

Important Notes-

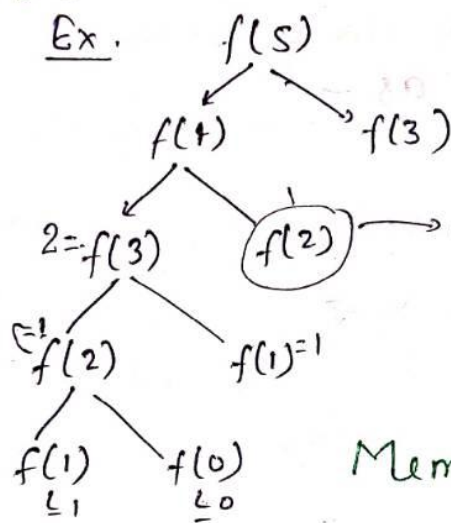
- 1) write the line `if(dp[i] != -1) return dp[i]` before calling the function when using memorization.
- 2) While declaring a variable in a recursive function, is absolutely OK, when the variable is being updated with recursive call.

```
\" C++  
  
int f(int i, int sum) {  
    int maxi = 0;  
  
    for (...) {  
        maxi = max(maxi, f(i+1, newSum));  
    }  
  
    return maxi;  
}
```

Dynamic programming — is a method used to solve complex problems by breaking them down into simpler subproblems. By solving each subproblem only once and storing the results.

- Two ways to implement it —
- Tabulation (bottom-up)
 - Memorization (top-down)

Fibonacci Number —



with recursion we will have to again do it $f(1)+f(0)$ but it is already calculated so, here is the concept of memorization comes.

Memorization — we tend to store the value of subproblem in some map / table.

dp =

-1	1	1	2	3	5
0	1	2	3	4	5

$dp[n+1] = \{-1\};$

```

f(n) {
    if (n <= 1) {
        return n;
    }
    if (dp[n] != -1) return dp[n];
    dp[n] = f(n-1) + f(n-2);
    return dp[n];
}
  
```

```

class Solution {
public:
    int dp[31]; // 0<=n<=31;
    int fib(int n) {
        if (n <= 1) return n;
        if (dp[n] != 0) return dp[n];
        else return dp[n] = fib(n-1) + fib(n-2);
    }
}
  
```

T.C = $O(N)$

S.C = $O(N) + O(N)$

recursion stacks

Recursion $\rightarrow$ Tabulation. (bottom-up).

(base case to the required)

$dp[n+1];$

$dp[0] = 0, dp[1] = 1;$

for(int $i = 2; i \leq n; i++$) {

$dp[i] = dp[i-1] + dp[i-2];$

}

TC = $O(N)$

SC = $O(N)$

we can reduce space complexity as -

$prev2 = 0, prev = 1;$

for($i = 2; i \leq n; i++$) {

$curi = prev + prev2;$

$prev2 = prev;$

$prev = curi;$

}

return prev;

to write a recursive funct

step ① suppose we have ^{to find} 'a'.

step ② let a function $f()$ can calculate it

step ③ search for the base condition.

step ③ to find some initial value write code and return the formula for those value in relation with function for next values.

Ex. $\text{sum}(n) = \text{sum}(n-1) + \text{func}(n-1)$

Climbing Stairs —

Each time you can climb one step or 2 step find the number of way you can reach

(Hints)

How to understand that it is a DP Problem —

- i) Count the total no. of ways.
- ii) Max/Min output.
- iii) Concept of all possible ways $\rightarrow$ count / best ways.

Short-cut trick.

- 1) Try to represent the problem in terms of Index.
- 2) Do all possible stuffs on that and according the problem statement.
- 3) If question says count all ways $\rightarrow$ sum of all stuffs.
for find min $\rightarrow$ min (of all stuffs).

f(ind) {

if (ind == 0) return 1;

if (ind == 1) return 1;

left = f(ind - 1);

right = f(ind - 2);

return left + right;

}

} Just like Fibonacci.

Frog Jump. (Frog can jump from n to $n+1$ or $n+2$ and Energy required will be height of $(n+1)^{\text{th}}$ stair - n^{th} stair.

Find the minimum energy required by the frog to reach from i^{th} stair to n^{th} stair

$\begin{array}{ccccccc} \text{F} & \xrightarrow{10} & \text{F} & \xrightarrow{10} & \text{F} & \xrightarrow{20} & \text{F} \\ 10 & & 20 & & 30 & & 10 \end{array}$
total = 40

Energy in going from zero to zero
 $f(0) = 0$

$f(\text{ind}) \{$

$\text{if}(\text{ind} == 0) \text{ return } 0;$

$\text{if}(\text{ind} == 1) \text{ return } 1;$

$\text{left} = f(\text{ind}-1) + \text{cost}(\text{arr}[\text{ind}] - \text{arr}[\text{ind}-1]);$

$\text{right} = f(\text{ind}-2) + \text{cost}(\text{arr}[\text{ind}] - \text{arr}[\text{ind}-2]);$

$\text{return}(\min(\text{left}, \text{right}));$

$\}$

→ jump as if there is only one stair then we cannot jump two.

Tabulation

~~$\text{int dp}[n]$~~

~~$\text{dp}[0] = 0;$~~

~~$\text{for}(i=1; i < n; i++) \{$~~

~~$\text{minSteps} = \text{INT_MAX};$~~

~~$\}$~~

Rod Cutting Problem

[L-29]

$N = 5 \rightarrow$ Rod length.

price $\rightarrow$

1	2	3	4	5
2	5	7	8	10

for length of the rod.

Try to pick length & sum them up to make N -
 $f(\text{ind}, N)$ { if $(\text{ind} == 0)$ return $(N \times \text{price}[0])$

int notTake = $0 + f(\text{ind} - 1, N)$;

int Take = INT_MIN ;

at the time N equals 0 then f will return notTake

rod-length = $\text{ind} + 1$;

if (rod-length $\leq N$) Take = $\text{price}[\text{ind}] + f(\text{ind}, N - \text{rod-length})$;

return $\max(\text{notTake}, \text{Take})$;

}

Recursion $\rightarrow$ Memorization

TC $\rightarrow O(\text{exponential})$

SC $= O(\text{target})$

ind, N.

$\downarrow$
 $[N] \times [N + 1]$

TC $\rightarrow O(N \times N)$.

SC $\rightarrow O(N \times N) \rightarrow$ no ideal.

Tabulation

1 $\rightarrow$ Base case.

2 $\rightarrow$ ind
N.

3 $\rightarrow$ copy the Recurrence

Frog Jump - II

if frog is allowed to jump from i^{th} index to $i+1, i+2, i+3 \dots (i+k)^{\text{th}}$ index. then what is the minimum energy required to reach the n^{th} index.

$f(\text{ind}) \{$

if ($\text{ind} == 0$) return 0;

if ($\text{dp}[\text{ind}] \neq -1$) return $\text{dp}[\text{ind}]$;

$\text{minSteps} = \text{INT_MAX}$;

for ($j=1; j \leq k; j++$) {

if ($\text{ind} \geq j$) {

$\text{jumpEnergy} = f(\text{ind}-j) + \text{abs}(a[\text{ind}] - a[\text{ind}-j])$

$\text{minSteps} = \min(\text{minSteps}, \text{jumpEnergy})$;

} else break;

return minSteps ;

}

T.C = $O(N \times K)$.

S.C = $O(1 + N)$

Tabulation

int $\text{dp}[n]$

$\text{dp}[0] = 0$

for ($i=1; i \leq n; i++$) {

$\text{minSteps} = \text{INT_MAX}$;

for ($j=1; j \leq k; j++$) {

if ($i-j \geq 0$) {

$\text{jump} = \text{dp}[i-j] + \text{abs}(a[i] - a[i-j])$

$\text{minSteps} = \min(\text{minSteps}, \text{jump})$;

}

$\text{dp}[i] = \text{minSteps}$;

}

}

Maximum Sum of Non-adjacent elements :

You are given an array of 'N' integers. You are supposed to return the maximum sum of the subsequence with a constraint that no two elements are adjacent in the given array list.

$f(\text{ind}) \{$

$\{ \text{if}(\text{ind} == 0) \text{return } a[\text{ind}];$

$\text{if}(\text{ind} < 0) \text{return } 0;$

$\text{if}(dp[\text{ind}] \neq -1) \text{return } dp[\text{ind}];$

$\text{pick} = a[\text{ind}] + f(\text{ind} - 2);$

$\text{notpick} = 0 + f(\text{ind});$

$\text{return } dp[\text{ind}] = \max(\text{pick}, \text{notpick});$

$\}$

for recursion
 $T.C = O(2^n)$

$S.C = O(N)$

for dp (memoization)

$T.C = O(N)$

$S.C = O(N) + O(N)$
 $= O(2N)$

recursion
stack
space.

Tabulation -

$dp[n];$
 $\text{dp}[0] = a[0];$
 $\text{int } neg = 0;$

$\text{for}(i = 1; i < n; i++) \{$

$\text{take} = a[i] + dp[i-2]; \text{if}(i > 1) \text{take} += dp[i-2];$

$\text{nontake} = 0 + dp[i-1];$

$dp[i] = \max(\text{take}, \text{nontake});$

$\}$

$T.C = O(N)$
 $S.C = O(N)$

198. House Robber

Attempted ☺

Medium

Topics

Companies

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return the *maximum amount of money you can rob tonight **without alerting the police***.

Example 1:

```
1 class Solution {
2 private:
3     int f(int i, vector<int>& nums, vector<int>& dp) {
4         if (i >= nums.size()) return 0; // Out-of-bounds case
5         if (dp[i] != -1) return dp[i]; // Memoized result
6
7         // Rob current house or skip it
8         int rob = nums[i] + f(i + 2, nums, dp);
9         int skip = f(i + 1, nums, dp);
10        return dp[i] = max(rob, skip);
11    }
12 public:
13     int rob(vector<int>& nums) {
14         vector<int> dp(nums.size(), -1); // Memoization array
15         return f(0, nums, dp); // Start from the first house
16     }
17 };
```

Space optimization -

or $\begin{cases} \text{prev2} = \text{prev} \\ \text{prev} = \text{curr} \end{cases}$

$\text{prev} = a[0];$

$\text{int prev2} = 0;$

$\text{for}(i=1; i < n; i++) \{$

$\text{take} = a[i]; \text{ if}(i \geq 1) \text{ take} += \text{prev2};$

$\text{nontake} = 0 + \text{prev};$

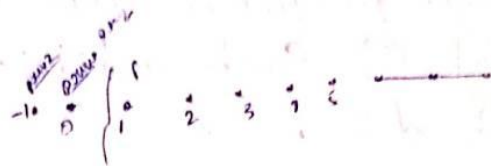
$\text{curr} = \max(\text{take}, \text{nontake});$

$\text{prev2} = \text{prev};$

$\text{prev} = \text{curr};$

$\}$

$\text{return prev};$



$T.C = O(N)$
 $S.C = O(1)$

746. Min Cost Climbing Stairs

Solved

Easy Topics Companies Hint

You are given an integer array `cost` where `cost[i]` is the cost of i^{th} step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

Input: `cost = [10,15,20]`

Output: 15

Explanation: You will start at index 1.

– Pay 15 and climb two steps to reach the top.

The total cost is 15.

Example 2:

```
1 class Solution{
2 private:
3     int f(vector<int>&cost,int n,vector<int>&dp){
4         if(n==0)return dp[0];
5         if(n==1)return dp[1];
6         if(dp[n]!=-1)return dp[n];
7         int onestep = cost[n]+f(cost,n-1,dp);
8         int twostep = cost[n] + f(cost,n-2,dp);
9         return dp[n] = min(onestep,twostep);
10    }
11 public:
12     int minCostClimbingStairs(vector<int>& cost) {
13         int n = cost.size();
14         vector<int>dp(n+1,-1);
15         dp[0] = cost[0];
16         dp[1] = cost[1];
17         return min(f(cost,n-1,dp),f(cost,n-2,dp));
18    }
19 };
```

Saved

Ln 8, Col 46

Ninja's Training (2-D concepts)

Ninja is planning this 'N' days-long training schedule. Each day he can perform only one of these 3 activities (Run, fight, new skill). Each activity has some merit points on each day.

You are given 2D array of size $N \times 3$ points with the points corresponding to each day and activity. Calculate the maximum number of merit points that ninja can earn.

greedy fails,

we need to explore all possible ways. so let's apply recursion.

1) Express problem in terms of index. let day is our index.

$f(\text{day}, \text{last})$ { we will solve from N^{th} day and 'last' will tell which activity is done out of '3'.

```

if (day == 0) {
    maxi = 0;
    for (i = 0; i < 3; i++) {
        if (i != last) {
            maxi = max(maxi, task[0][i]);
        }
    }
    return maxi;
}
if (dp[day][last] != -1) return dp[day][last];
maxi = 0;
for (i = 0; i < 3; i++) {
    if (i != last) {
        points = task[day][i] + f(day-1, i);
        maxi = max(maxi, points);
    }
}
dp[day][last] = maxi;
return maxi;
    
```

as in each day there is total 4 possible cases 0, 1, 2, 3. so we need dp of $N \times 4$.

Tabulation:-

```
int dp[n][4]
dp[0][0] = max(arr[0][1], arr[0][2])
dp[0][1] = max(arr[0][0], arr[0][2])
dp[0][2] = max(arr[0][0], arr[0][1])
dp[0][3] = max(arr[0][0], arr[0][1], arr[0][2])
```

```
for(day = 1 → n-1) {
```

```
    for(lasttask = 0 → 3) {
```

```
        dp[day][last] = 0;
```

```
        int maxi = 0;
```

```
        for(int task = 0; task < 3; task++) {
```

```
            if(task != last) {
```

```
                int point = arr[day][task] + dp[day-1][task];
```

```
                maxi = max(maxi, point);
```

```
                dp[day][last] = dp[day][last];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
return dp[n-1][3];
```

Space optimization:-

Cr- On grids / DP on matrix -

1) Total unique paths:-

You are present at point 'A' which is top-left cell of an $M \times N$ matrix, your destination is point 'B', which is bottom right cell of same matrix. Your path is to find the total number of unique paths from point 'A' to point 'B'. In other words you will be given the dimension of the matrix as integers 'M' and 'N'. Your task is to find the total number of unique paths from the cell $matrix[0][0]$ to $matrix[M-1][N-1]$.

To traverse in the matrix, you can move right or down at each step. For example in a given point $matrix[i][j]$ you can move to either $matrix[i+1][j]$ or $matrix[i][j+1]$.

Recursion -

$f(i, j)$ {

if $(i=0 \ \& \ j=0)$ {

return 1;

}

if $(i < 0 \ || \ j < 0)$ return 0;

if $(dp[i][j] \neq -1)$ return $dp[i][j]$;

up = $f(i-1, j)$;

left = $f(i, j-1)$;

return $dp[i][j] = up + left$;

★ While counting ways return 1 if you have reached your destination and return 0 if not.

T.C = $2^{m \times n}$.

(as for all the cell we have two paths).

S.C = $O((m-1) + (n-1))$.

Memoization - $TC = O(m \times n)$.

Parameters, which are changing are i, j . So we have to store them, max of i is $n-1$ & max of j is $m-1$ so

Step 1 declare a dp of size $(n-1) \times (m)$.

Step 2. Store the answer that you are returning

Step 3 Before any function call

check for $dp[i][j] \neq -1$ if not then return $dp[i][j]$

Tabulation -

$dp[0][0] = 1;$

for ($i = 0 \rightarrow m-1$) {

for ($j = 0 \rightarrow n-1$) {

if ($i=0$ & $j=0$) continue;

if ($i > 0$)

up = $dp[i-1][j];$

if ($j > 0$)

left = $dp[i][j-1];$

$dp[i][j] = up + left;$

}

}

★ Recursion is TOP-DOWN
as the top guy gets
answer by going down.
→ Tabulation is bottom-
up. as the bottom guy
gets answer by going up.

space optimization - if there is a previous row and previous
column then we can definitely optimize the space.
 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$



→ To compute this we need
These, some need to just store
the previous row.
 $pow[n] \rightarrow 0$ (for previous row initialize).

```
for (int i = 0; i < n; i++) {  
    temp[n] = 0;  
    for (int j = 0; j < m; j++) {  
        if (i == 0 & j == 0) {  $dp[i][j] = 1;$  }  
        else {  
             $temp[j] = pow[j] + temp[j-1];$   
        }  
    }  
    pow = temp;  
}
```

memoization:-

```
1 int f(int i, int j, vector<vector<int>> &dp) {
2     if(i == 0 && j == 0) return 1;
3     if(i < 0 || j < 0) return 0;
4     if(dp[i][j] != -1) return dp[i][j];
5     int up = f(i-1, j, dp);
6     int left = f(i, j-1, dp);
7     return dp[i][j] = up + left;
8 }
9 int uniquePaths(int m, int n) {
10     vector<vector<int>> dp(m, vector<int>(n, -1));
11     return f(m-1, n-1, dp);
}
```

Tabulation

```
int f(int i, int j, vector<vector<int>> &dp) {
    if(i == 0 && j == 0) return 1;
    if(i < 0 || j < 0) return 0;

    int dp[m][n];
    for(int i = 0; i < m; i++) {
        for(int j = 0; j < n; j++) {
            if(i == 0 && j == 0) dp[i][j] = 1;
            else {
                int up = 0;
                int left = 0;
                if(i > 0) up = dp[i-1][j];
                if(j > 0) left = dp[i][j-1];
                dp[i][j] = up + left;
            }
        }
    }

    return dp[m-1][n-1];
}
```

Space Optimization

```
int uniquePaths(int m, int n) {  
    vector<int> prev(n, 0);  
    for(int i = 0; i < m; i++) {  
        vector<int> cur(n, 0);  
        for(int j = 0; j < n; j++) {  
            if(i == 0 && j == 0) cur[j] = 1;  
            else {  
                int up = 0;  
                int left = 0;  
                if(i > 0) up = prev[j];  
                if(j > 0) left = cur[j-1];  
                dp[i][j] = up + left;  
            }  
        }  
        prev = cur;  
    }  
    return prev[n-1];  
}
```

Ques-

Problem Statement

[Suggest Edit](#)

Given a 'N' * 'M' maze with obstacles, count and return the number of paths to reach the right-bottom cell from the top-left cell. A cell in the given maze has a value -1 if it is a blockage or dead-end, else 0. From a given cell, we are allowed to move to cells (i+1, j) and (i, j+1) only. Since the answer can be large, print it modulo $10^9 + 7$.

Recursion-

$f(i, j)$ { last indices of 2-D array
if $i > 0$ && $j > 0$ && $nums[i][j] == -1$
return 0;
if $(i == 0 \text{ \& \& } j == 0)$ return 1;
if $(i < 0 \text{ || } j < 0)$ return 0;
up = $f(i-1, j)$;
left = $f(i, j-1)$; } return left + up;

Ques -

Problem Statement

[Suggest Edit](#)

Ninjabland is a country in the shape of a 2-Dimensional grid 'GRID', with 'N' rows and 'M' columns. Each point in the grid has some cost associated with it.

Find a path from top left i.e. (0, 0) to the bottom right i.e. ('N' - 1, 'M' - 1) which minimizes the sum of the cost of all the numbers along the path. You need to tell the minimum sum of that path.

Note:

You can only move down or right at any point in time.

last index i.e. (m-1, n-1)

Recursion - $f(i, j)$ {
if (i == 0 && j == 0) return arr[0][0];
if (i < 0 || j < 0) return INT_MAX;
up = arr[i][j] + f(i-1, j);
left = arr[i][j] + f(i, j-1);
return min(up, left);
}

Memorization:

```
1 int f(int i, int j, vector<vector<int>> &grid, vector<vector<int>> &dp) {  
2     if(i == 0 && j == 0) return grid[i][j];  
3     if(i < 0 || j < 0) return 1e9;  
4     if(dp[i][j] != -1) return dp[i][j];  
5     int up = grid[i][j] + f(i-1, j, grid, dp);  
6     int left = grid[i][j] + f(i, j-1, grid, dp);  
7     * return dp[i][j] = min(left, up) + grid[i][j];  
8 }
```

```
9 int minSumPath(vector<vector<int>> &grid) {  
10     int n = grid.size();  
11     int m = grid[0].size();  
12     vector<vector<int>> dp(n, vector<int> (m, -1));  
13     return f(n-1, m-1, grid, dp);  
14 }
```

Tabulation:-

```
9 int minSumPath(vector<vector<int>> &grid) {
10     int n = grid.size();
11     int m = grid[0].size();
12     vector<vector<int>> dp(n, vector<int>(m, 0));
13     for(int i = 0; i < n; i++) {
14         for(int j = 0; j < m; j++) {
15             if(i == 0 && j == 0) dp[i][j] = grid[i][j];
16             else {
17                 int up = grid[i][j];
18                 if(i > 0) up += dp[i-1][j];
19                 else up += 1e9;
20
21                 int left = grid[i][j];
22                 if(j > 0) left += dp[i][j-1];
23                 else left += 1e9;
24
25                 dp[i][j] = min(left, up);
26             }
27         }
28     }
29     return dp[n-1][m-1];
```

Space optimization -

```
vector<vector<int>> up(n, vector<int>(m, 0));  
vector<int> prev(m, 0);  
// i = n  
for(int i = 0; i < n; i++) {  
    vector<int> cur(m, 0);  
    for(int j = 0; j < m; j++) {  
        if(i == 0 && j == 0) cur[j] = grid[i][j];  
        else {  
            int up = grid[i][j];  
            //requiring previous row's j column  
            if(i > 0) up += prev[j];  
            else up += 1e9;  
  
            int left = grid[i][j];  
            //current row's j-1 column  
            if(j > 0) left += cur[j-1];  
            else left += 1e9;  
  
            cur[j] = min(left, up);  
        }  
    }  
    prev = cur;  
}
```

```
29  
30         cur[j] = min(left, up);  
31     }  
32 }  
33 prev = cur;  
34 }  
35 return prev[m-1];  
36 }
```

Problem Statement

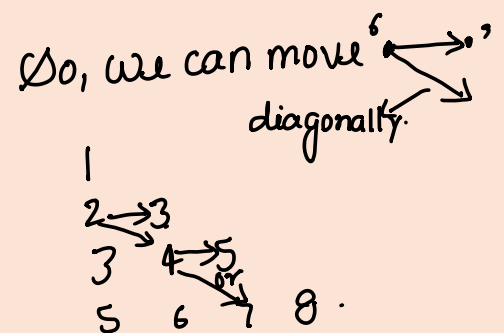
[Suggest Edit](#)

You are given a triangular array/list 'TRIANGLE'. Your task is to return the minimum path sum to reach from the top to the bottom row.

The triangle array will have N rows and the i-th row, where $0 \leq i < N$ will have $i + 1$ elements.

You can move only to the adjacent number of row below each step. For example, if you are at index j in row i, then you can move to i or i + 1 index in row j + 1 in each step.

For Example :



Ques -

Recursion - $f(0,0)$ {

```
if(i==n-1) return arr[i][j];  
int pickdown = arr[i][j] + f(i+1, j);  
int pickdiagonal = arr[i][j] + f(i+1, j+1);  
return min(pickdown, pickdiagonal);  
}
```

Note:- In general if recursion is from 0 to $n-1$ then tabulation will be from $n-1$ to 0.

Memorization

```
1 int f(int i, int j, vector<vector<int>>& triangle, int n,  
2     vector<vector<int>>& dp) {  
3     if(i==n-1) {  
4         return triangle[n-1][j];  
5     }  
6     if(dp[i][j] != -1) return dp[i][j];  
7     int d = triangle[i][j] + f(i+1, j, triangle, n, dp);  
8     int dg = triangle[i][j] + f(i+1, j+1, triangle, n, dp);  
9     return dp[i][j] = min(d, dg);  
10 }  
11 int minimumPathSum(vector<vector<int>>& triangle, int n){  
12     vector<vector<int>> dp(n, vector<int>(n, -1));  
13     return f(0,0, triangle, n, dp);  
14 }
```

Tabulation

```
11 int minimumPathSum(vector<vector<int>>& triangle, int n){  
12     * vector<vector<int>> dp(n, vector<int>(n, 0));  
13     for(int j = 0; j<n; j++) dp[n-1][j] = triangle[n-1][j];  
14     for(int i = n-2; i>=0; i--) {  
15         for(int j = i; j>=0; j--) {  
16             int d = triangle[i][j] + dp[i+1][j];  
17             int dg = triangle[i][j] + dp[i+1][j+1];  
18             dp[i][j] = min(d, dg);  
19         }  
20     }  
21     return dp[0][0];  
22 }
```

Space optimization - $\rightarrow$

1
2 3
This row is cur at start $\rightarrow$ 3 4 5
This row is front at start $\rightarrow$ 6 7 8 9
 $\leftarrow$ So we need to store only the current row and the previous to calculate $dp[i][j]$, here we will proceed from bottom as done in tabulation.

Another approach for space optimization -

```
12 ✓ int minimumTotal(vector<vector<int>>& triangle) {  
13     int n = triangle.size();  
14     vector<int>prev(n,1e8);  
15 ✓     for(int i = 0; i<n; i++){  
16         vector<int>cur(n,1e8);  
17 ✓         for(int j = 0; j<=i; j++){  
18             if(i==0 && j==0)cur[j] = triangle[i][j];  
19 ✓             else{  
20                 int up = 1e8, diag = 1e8;  
21                 if(i>0 && j<=i)up = triangle[i][j] + prev[j];  
22                 if(i>0 && j>0 && j<=i)diag = triangle[i][j] + prev[j-1];  
23                 cur[j] = min(up,diag);  
24             }  
25         }  
26         prev = cur;  
27     }  
28     int mini = INT_MAX;  
29 ✓     for(int i = 0; i<n; i++){  
30         mini = min(mini,prev[i]);  
31     }  
32     return mini;  
33 }  
34 };
```

for memoization -

```
3 ✓ int f(vector<vector<int>>&triangle,int i, int j,vector<vector<int>>&dp){  
4     if(i==0 && j == 0)return triangle[i][j];  
5     if(i<0 || j<0 || j>i) return 1e8;  
6     if(dp[i][j]!=-1)return dp[i][j];  
7     int up = triangle[i][j] + f(triangle,i-1,j,dp) ;  
8     int diag = triangle[i][j] + f(triangle,i-1,j-1,dp);  
9     return dp[i][j] = min(up,diag);  
10 }
```

3-D DP -

Ques-

Problem Statement

Suggest Edit

You have been given an $N \times M$ matrix filled with integer numbers, find the maximum sum that can be obtained from a path starting from any cell in the first row to any cell in the last row.

From a cell in a row, you can move to another cell directly below that row, or diagonally below left or right. So from a particular cell (row, col), we can move in three directions i.e.

```
Down: (row+1,col)
Down left diagonal: (row+1,col-1)
Down right diagonal: (row+1,col+1)
```

★
Recursion -

Base cases are — i) Destination
ii) out of bound indices

$f(i, j, a)$

if ($j < 0$ || $j > m-1$) return -1 eg ← as we need max so we want these invalid case to be min. so that they are easily rejected.

if ($i == 0$) return $a[i][j]$

int diagonalleft = $a[i][j] + f(i-1, j-1)$;

int up = $a[i][j] + f(i-1, j)$;

int diagonalright = $a[i][j] + f(i-1, j+1)$;

return max(diagonalleft, up, diagonalright);

}

int maxsum(arr) {

int maxi = -1e9

for (int j = 0; j < n; j++) {

maxi = max(maxi, $f(i, j, a)$)

}

return maxi;

}



Memorization:-

```
1 int f(int i, int j, vector<vector<int>> &matrix, vector<vector<int>> &dp) {
2     if(j<0 || j >= matrix[0].size()) return -1e8;
3     if(i == 0) return matrix[0][j];
4     if(dp[i][j] != -1) return dp[i][j];
5     int u = matrix[i][j] + f(i-1, j, matrix, dp);
6     int ld = matrix[i][j] + f(i-1, j-1, matrix, dp);
7     int rd = matrix[i][j] + f(i-1, j+1, matrix, dp);
8
9     return dp[i][j] = max(u, max(ld, rd));
10 }
```

```
11 int getMaxPathSum(vector<vector<int>> &matrix)
12 {
13     int n = matrix.size();
14     int m = matrix[0].size();
15     vector<vector<int>> dp(n, vector<int>(m, -1));
16     int maxi = -1e8;
17     for(int j = 0; j<m; j++) {
18         maxi = max(maxi, f(n-1, j, matrix, dp));
19     }
20     return maxi;
21 }
```

Tabulation -

```
15     vector<vector<int>> dp(n, vector<int>(m, 0));
16
17     for(int j = 0; j<m; j++) dp[0][j] = matrix[0][j];
18     for(int i = 1; i<n; i++) {
19         for(int j = 0; j<m; j++) {
20             int u = matrix[i][j] + dp[i-1][j];
21             int ld = matrix[i][j];
22             if(j-1>=0) ld += dp[i-1][j-1];
23             else ld += -1e8;
24             int rd = matrix[i][j];
25             if(j+1<m) rd += dp[i-1][j+1];
26             else rd += -1e8;
27             dp[i][j] = max(u, max(ld, rd));
28         }
29     }
```

```
32     int maxi = -1e8;
33     for(int j = 0; j<m; j++) {
34         maxi = max(maxi, dp[n-1][j]);
35     }
36     return maxi;
37 }
```

Ques*

931. Minimum Falling Path Sum

Solved ✓

Medium

Topics

Companies

Given an $n \times n$ array of integers `matrix`, return the **minimum sum** of any **falling path** through `matrix`.

A **falling path** starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position `(row, col)` will be `(row + 1, col - 1)`, `(row + 1, col)`, or `(row + 1, col + 1)`.

Tabulation-

```

4 private:
5     int minipath(vector<vector<int>>&matrix,int n){
6         vector<vector<int>>dp(n,vector<int>(n));
7         for(int i = n-1; i>=0; i--){
8             for(int j = 0; j<n; j++){
9                 if(i==n-1)dp[i][j] = matrix[i][j];
10                else{
11
12                    int d1 = matrix[i][j];
13                    if(j>0 && i<n-1) d1 += dp[i+1][j-1];
14                    else d1 += 1e9;
15                    int b = matrix[i][j];
16                    if(i<n-1) b+= dp[i+1][j];
17                    else b += 1e9;
18                    int dr = matrix[i][j];
19                    if(i<n-1 && j<n-1) dr += dp[i+1][j+1];
20                    else dr += 1e9;
21                    int mini = min({d1,b,dr});
22                    dp[i][j] = mini;
23                }
24            }
25        }
26    }
27    int minimal = INT_MAX;
28    for(int j = 0; j<n; j++){
29        minimal = min(minimal,dp[0][j]);
30    }
31    return minimal;
32 }
33
34 public:

```

```

34 public:
35     int minFallingPathSum(vector<vector<int>>& matrix) {
36         int n = matrix.size();
37         return minipath(matrix, n);
38     }
39 }
40 };

```

Ques

Problem Statement

Suggest Edit

Ninja has a 'GRID' of size 'R' X 'C'. Each cell of the grid contains some chocolates. Ninja has two friends Alice and Bob, and he wants to collect as many chocolates as possible with the help of his friends.

Initially, Alice is in the top-left position i.e. (0, 0), and Bob is in the top-right place i.e. (0, 'C' - 1) in the grid. Each of them can move from their current cell to the cells just below them. When anyone passes from any cell, he will pick all chocolates in it, and then the number of chocolates in that cell will become zero. If both stay in the same cell, only one of them will pick the chocolates in it.

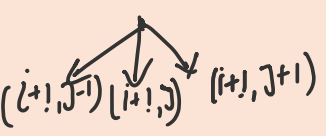
If Alice or Bob is at (i, j) then they can move to (i + 1, j), (i + 1, j - 1) or (i + 1, j + 1). They will always stay inside the 'GRID'.

Your task is to find the maximum number of chocolates Ninja can collect with the help of his friends by following the above rules.

Example:

Recursion

$f(i_1, j_1, j_2)$
 as bob and alice will always be in the same row.
 as returning IN PM will lead to problem as it will be added to other values later.
 if ($j_1 < 0 || j_1 > m-1 || j_2 < 0 || j_2 > m-1$) return $1e-8$.
 if ($i == n-1$) {
 if ($j_1 == j_2$) return $a[i][j_1]$;
 else return $a[i][j_1] + a[i][j_2]$;
 }
 // Explore all the paths alice and bob can move.
 So, for each movement of alice bob also has 3 options of movement.
 max_i = 0;
 for ($dj_1 = -1 \rightarrow 1$) {
 for ($dj_2 = -1 \rightarrow 1$) {
 if ($j_1 == j_2$) max_i = $\max(\max_i, a[i][j_1] + f(i+1, j_1+dj_1, j_2+dj_2))$;
 else max_i = $\max(\max_i, a[i][j_1] + a[i][j_2] + f(i+1, j_1+dj_1, j_2+dj_2))$;
 }
 }
 return max_i;
}



Memorization

```
int maxChocoUtil(int i, int j1, int j2, int n, int m, vector<vector<int>>> &grid,
vector<vector<vector<int>>>> &dp) {

    // Check if the positions (j1, j2) are valid
    if (j1 < 0 || j1 >= m || j2 < 0 || j2 >= m)
        return -1e9; // A very large negative value to represent an invalid position

    // Base case: If we are at the last row, return the chocolate at the positions (j1, j2)
    if (i == n - 1) {
        if (j1 == j2)
            return grid[i][j1];
        else
            return grid[i][j1] + grid[i][j2];
    }

    // If the result for this state is already computed, return it
    if (dp[i][j1][j2] != -1)
        return dp[i][j1][j2];

    int maxi = INT_MIN;

    // Try all possible moves (up, left, right) for both positions (j1, j2)
    for (int di = -1; di <= 1; di++) {
        for (int dj = -1; dj <= 1; dj++) {
            int ans;
            if (j1 == j2)
                ans = grid[i][j1] + maxChocoUtil(i + 1, j1 + di, j2 + dj, n, m, grid, dp);
            else
                ans = grid[i][j1] + grid[i][j2] + maxChocoUtil(i + 1, j1 + di, j2 + dj, n, m, grid, dp);

            // Update the maximum result
            maxi = max(maxi, ans);
        }
    }
}
```



```

    }
}

// Store the maximum result for this state in dp
return dp[i][j1][j2] = maxi;
}

// Function to find the maximum chocolates that can be collected
int maximumChocolates(int n, int m, vector<vector<int>> &grid) {
    // Create a 3D DP array to store computed results
    vector<vector<vector<int>>> dp(n, vector<vector<int>>(m, vector<int>(m, -1)));

    // Call the recursive utility function to find the maximum chocolates starting from the first row
    return maxChocoUtil(0, 0, m - 1, n, m, grid, dp);
}

```

Tabulation -

```

int maximumChocolates(int n, int m, vector<vector<int>> &grid) {
    // Create a 3D DP array to store computed results
    vector<vector<vector<int>>> dp(n, vector<vector<int>>(m, vector<int>(m, 0)));

    // Initialize the DP array for the last row
    for (int j1 = 0; j1 < m; j1++) {
        for (int j2 = 0; j2 < m; j2++) {
            if (j1 == j2)
                dp[n - 1][j1][j2] = grid[n - 1][j1];
            else
                dp[n - 1][j1][j2] = grid[n - 1][j1] + grid[n - 1][j2];
        }
    }
}

```

} base case of tabulation.

// Outer nested loops for traversing the DP array from the second-to-last row up to the first row

```
for (int i = n - 2; i >= 0; i--) {  
    for (int j1 = 0; j1 < m; j1++) {  
        for (int j2 = 0; j2 < m; j2++) {  
            int maxi = INT_MIN;  
  
            // Inner nested loops to try out 9 options (diagonal moves)  
            for (int di = -1; di <= 1; di++) {  
                for (int dj = -1; dj <= 1; dj++) {  
                    int ans;  
  
                    if (j1 == j2)  
                        ans = grid[i][j1];  
                    else  
                        ans = grid[i][j1] + grid[i][j2];  
  
                    // Check if the move is valid (within the grid boundaries)  
                    if ((j1 + di < 0 || j1 + di >= m) || (j2 + dj < 0 || j2 + dj >= m))  
                        ans += -1e9; // A very large negative value to represent an invalid move  
                    else  
                        ans += dp[i + 1][j1 + di][j2 + dj]; // Include the DP result from the next row  
                    maxi = max(ans, maxi); // Update the maximum result  
                }  
            }  
  
            dp[i][j1][j2] = maxi; // Store the maximum result for this state in the DP array  
        }  
    }  
}
```

// The maximum chocolates that can be collected is stored at the top-left corner of the DP array

```
return dp[0][0][m - 1];
```

```
}
```

Space Optimization → 3D → 2D.

```
int maximumChocolates(int n, int m, vector<vector<int>> &grid) {
```

```
    // Create two 2D vectors 'front' and 'cur' to store computed results
```

```
    vector<vector<int>> front(m, vector<int>(m, 0));
```

```
    vector<vector<int>> cur(m, vector<int>(m, 0));
```

```
    // Initialize 'front' for the last row
```

```
    for (int j1 = 0; j1 < m; j1++) {
```

```
        for (int j2 = 0; j2 < m; j2++) {
```

```
            if (j1 == j2)
```

```
                front[j1][j2] = grid[n - 1][j1];
```

```
            else
```

```
                front[j1][j2] = grid[n - 1][j1] + grid[n - 1][j2];
```

```
        }
```

```
    }
```

```
    // Outer nested loops for traversing the DP array from the second-to-last row up to the first row
```

```
    for (int i = n - 2; i >= 0; i--) {
```

```
        for (int j1 = 0; j1 < m; j1++) {
```

```
            for (int j2 = 0; j2 < m; j2++) {
```

```
                int maxi = INT_MIN;
```

```
                // Inner nested loops to try out 9 options (diagonal moves)
```

```
                for (int di = -1; di <= 1; di++) {
```

```
                    for (int dj = -1; dj <= 1; dj++) {
```

```
                        int ans;
```

```
                        if (j1 == j2)
```

```
                            ans = grid[i][j1];
```

```

else
    ans = grid[i][j1] + grid[i][j2];
    // Check if the move is valid (within the grid boundaries)
    if ((j1 + di < 0 || j1 + di >= m) || (j2 + dj < 0 || j2 + dj >= m))
        ans += -1e9; // A very large negative value to represent an invalid move
    else
        ans += front[j1 + di][j2 + dj]; // Include the value from the 'front' array
    maxi = max(ans, maxi); // Update the maximum result
}
}
cur[j1][j2] = maxi; // Store the maximum result for this state in the 'cur' array
}
}
front = cur; // Update 'front' with the values from 'cur' for the next iteration
}
// The maximum chocolates that can be collected is stored at the top-left corner of the
'front' array
return front[0][m - 1];
}

```


DP ON SUBSEQUENCES-

↳ any contiguous/non-contiguous part of an array but in the same order.

Problem Statement

[Suggest Edit](#)

You are given an array/list 'ARR' of 'N' positive integers and an integer 'K'. Your task is to check if there exists a subset in 'ARR' with a sum equal to 'K'.

Note: Return true if there exists a subset with sum equal to 'K'. Otherwise, return false.

For Example :

If 'ARR' is {1,2,3,4} and 'K' = 4, then there exists 2 subsets with sum = 4. These are {1,3} and {4}. Hence, return true.

Key points to remember-

- 1) Express in terms of index & target.
- 2) Explore possibilities of that index.
 - i) that index is part of the subsequence.
 - ii) that index isn't part of the subsequence.

→ $f(n-1, \text{target})$ → means in the entire array, does there exist subsequence of sum = target.

- 3) In all the subsequence problem use the concepts — take & not-take

Recursive Soln-

```
1 bool f(int ind, int target, vector<int> &arr) {  
2     if(target == 0) return true;  
3     if(ind == 0) return (a[0] == target);  
4  
5     bool notTake = f(ind - 1, target, arr);  
6     bool take = false;  
7     if(a[ind] <= target) take = f(ind - 1, target - arr[ind], arr);  
8     return take | notTake;  
9 }
```

Memoization:-

```
1 bool f(int ind, int target, vector<int> &arr, vector<vector<int>> &dp)
2     if(target == 0) return true;
3     if(ind == 0) return (arr[0] == target);
4     if(dp[ind][target] != -1) return dp[ind][target];
5     bool notTake = f(ind - 1, target, arr, dp);
6     bool take = false;
7     if(arr[ind] <= target) take = f(ind - 1, target - arr[ind], arr, dp);
8     return dp[ind][target] = take | notTake;
9 }
10 bool subsetSumToK(int n, int k, vector<int> &arr) {
11     vector<vector<int>> dp(n, vector<int>(k+1, -1));
12     return f(n-1, k, arr, dp);
13     // Write your code here.
14 }
```

Tabulation-

```
10 bool subsetSumToK(int n, int k, vector<int> &arr) {
11     vector<vector<bool>> dp(n, vector<bool>(k+1, 0));
12     for(int i = 0; i < n; i++) dp[i][0] = true;
13     dp[0][arr[0]] = true;
14     for(int ind = 1; ind < n; ind++) {
15         for(int target = 1; target <= k; target++) {
16             bool notTake = dp[ind - 1][target];
17             bool take = false;
18             if(arr[ind] <= target) take = dp[ind - 1][target - arr[ind]];
19             dp[ind][target] = take | notTake;
20         }
21     }
22     return dp[n-1][k];
23     // Write your code here.
```

Ques-

You are given an array 'ARR' of 'N' positive integers. Your task is to find if we can partition the given array into two subsets such that the sum of elements in both subsets is equal.

For example, let's say the given array is [2, 3, 3, 3, 4, 5], then the array can be partitioned as [2, 3, 5], and [3, 3, 4] with equal sum 10.

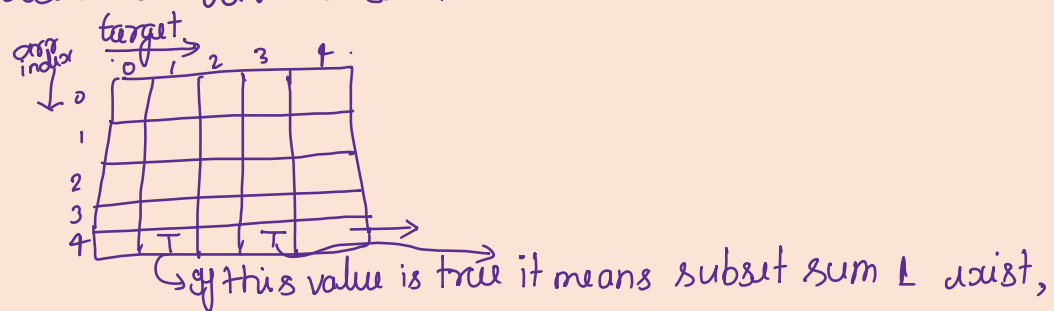
Follow Up:

Can you solve this using not more than $O(S)$ extra space, where S is the sum of all elements of the given array?

→ So, Basically we need to figure out whether there is a subset whose sum is equal to the $\frac{S}{2}$. where S is sum of the array elements.

Partition a set into two subset such that the difference of subset sum is minimum-(We have to find the minimum difference)-

difference)- Using tabulation method, suppose we have to find whether subset sum of 12 exist or not, then tabulation method also gives us that subset sum of 1 to 12 exist or not. So, suppose we have sum of the elements of array as S , then the subset sum(s_i) for which $(s_i - (S - s_i))$ is minimum will be our answer.



```
7     for(int i = 0; i < n; i++) dp[i][0] = true;
8     if(arr[0] <= k) dp[0][arr[0]] = true;
9     for(int ind = 1; ind < n; ind++) {
10        for(int target = 1; target <= k; target++) {
11            bool notTake = dp[ind - 1][target];
12            bool take = false;
13            if(arr[ind] <= target) take = dp[ind - 1][target - arr[ind]];
14            dp[ind][target] = take | notTake;
15        }
16    }
17
18    // dp[n-1][col->0...totsum]
19    int mini = 1e9;
20    for(int s1 = 0; s1 <= totSum/2; s1++) {
21        if(dp[n-1][s1] == true) {
22            mini = min(mini, abs((totSum - s1) - s1));
23        }
24    }
25    return mini;
```

Ques-

You are given an array (0-based indexing) of positive integers and you have to tell how many different ways of selecting the elements from the array are there such that the sum of chosen elements is equal to the target number "tar".

Note:

Two ways are considered different if sets of indexes of elements chosen by these ways are different.

Input is given such that the answer will fit in a 32-bit integer.

Recursion-

```
1 int f(int ind, int sum, vector<int> &num) {  
2     if(sum == 0) return 1;  
3     if(ind == 0) return num[0] == sum;  
4  
5     int notTake = f(ind-1, sum);  
6     int take = 0;  
7     if(num[ind] <= sum) take = f(ind - 1, sum - num[ind]);  
8  
9     return notTake + take;  
10 }  
11 int findWays(vector<int> &num, int tar)  
12 {  
13     int n = num.size();  
14     return f(n-1, tar, num);  
15 }
```

Memorization

```
1 int f(int ind, int sum, vector<int> &num, vector<vector<int>> &dp) {
2     if(sum == 0) return 1;
3     if(ind == 0) return num[0] == sum;
4     if(dp[ind][sum] != -1) return dp[ind][sum];
5     int notTake = f(ind-1, sum, num, dp);
6     int take = 0;
7     if(num[ind] <= sum) take = f(ind - 1, sum - num[ind], num, dp);
8
9     return dp[ind][sum] = notTake + take;
10 }
11 int findWays(vector<int> &num, int tar)
12 {
13     int n = num.size();
14     vector<vector<int>> dp(n, vector<int>(tar+1, -1));
15
16     return f(n-1, tar, num, dp);
17 }
```

Tabulation

```
7 int findWays(vector<int> &num, int tar)
8 {
9     int n = num.size();
10    vector<vector<int>> dp(n, vector<int>(tar+1, 0));
11    for(int idx = 0; i<n; i++) dp[i][0] = 1;
12    if(a[0] <= tar) dp[0][a[0]] = 1;
13    for(int i = 1; i<n; i++) {
14        for(int sum = 0; sum <= tar; sum++) {
15            int notTake = dp[ind-1][sum];
16            int take = 0;
17            if(num[ind] <= sum) take = dp[ind - 1][sum - num[ind]];
18
19            dp[ind][sum] = notTake + take;
20        }
21    }
22    return dp[n-1][tar];
23 }
```


Space-optimization-

```
7 int findWays(vector<int> &num, int tar)
8 {
9     int n = num.size();
10    vector<int> prev(tar+1, 0), cur(tar+1);
11    prev[0] = cur[0] = 1;
12    if(num[0] <= tar) prev[num[0]] = 1;
13
14    for(int ind = 1; ind < n; ind++) {
15        for(int sum = 0; sum <= tar; sum++) {
16            int notTake = prev[sum];
17            int take = 0;
18            if(num[ind] <= sum) take = prev[sum - num[ind]];
19
20            cur[sum] = notTake + take;
21        }
22        prev = cur;
23    }
24    return prev[tar];
25 }
```

Ques-

 0

Partitions With Given Difference

Difficulty: **MEDIUM**

 Contributed By
Abhishek Bansal | Level 1

Problem Statement [Suggest Edit](#)

Given an array 'ARR', partition it into two subsets (possibly empty) such that their union is the original array. Let the sum of the elements of these two subsets be 'S1' and 'S2'.

Given a difference 'D', count the number of partitions in which 'S1' is greater than or equal to 'S2' and the difference between 'S1' and 'S2' is equal to 'D'. Since the answer may be too large, return it modulo '10⁹ + 7'.

If 'Pi_Sj' denotes the Subset 'j' for Partition 'i'. Then, two partitions P1 and P2 are considered different if:

$$S_1 - S_2 = D \Rightarrow S_1 - (S - S_1) = D \Rightarrow S_1 = \frac{D + S}{2}$$

So, it can also be solved using the previous² method. ← total sum

0/1 knapsack Problems -

 **0 1 Knapsack**

2 Difficulty: **EASY**

 Contributed By **Anup Kumar Singh**

Level 1

Avg. time to solve **15 min**

Success Rate **85%**



Problem Statement [Suggest Edit](#)

A thief is robbing a store and can carry a maximal weight of W into his knapsack. There are N items and the i th item weighs w_i and is of value v_i . Considering the constraints of the maximum weight that a knapsack can carry, you have to find and return the maximum value that a thief can generate by stealing items.

Recursion-

```
1 int f(int ind, int W, vector<int> &wt, vector<int> &val) {
2     if(ind == 0) {
3         if(wt[0] <= W) return val[0];
4         return 0;
5     }
6
7     int notTake = 0 + f(ind - 1, W, wt, val);
8     int take = INT_MIN;
9     if(wt[ind] <= W) {
10         take = val[ind] + f(ind-1, W - wt[ind], wt, val);
11     }
12     return max(take, notTake);
13 }
14 int knapsack(vector<int> weight, vector<int> value, int n, int maxWeight
15 {
16     return f(n-1, maxWeight, weight, value);
17     // Write your code here
18 }
```

Memorization -

```
1 int f(int ind, int W, vector<int> &wt, vector<int> &val,
2     vector<vector<int>> &dp) {
3     if(ind == 0) {
4         if(wt[0] <= W) return val[0];
5         return 0;
6     }
7     if(dp[ind][W] != -1) return dp[ind][W];
8     int notTake = 0 + f(ind - 1, W, wt, val, dp);
9     int take = INT_MIN;
10    if(wt[ind] <= W) {
11        take = val[ind] + f(ind-1, W - wt[ind], wt, val, dp);
12    }
13    return dp[ind][W] = max(take, notTake);
14 }
15 int knapsack(vector<int> weight, vector<int> value, int n, int maxWeight)
16 {
17     vector<vector<int>> dp(n, vector<int> (maxWeight + 1, -1));
18     return f(n-1, maxWeight, weight, value, dp);
19     // Write your code here
20 }
```

Space Optimization -

```
15 int knapsack(vector<int> wt, vector<int> val, int n, int maxWeight)
16 {
17     vector<vector<int>> dp(n, vector<int> (maxWeight + 1, 0));
18     for(int W = wt[0]; W <= maxWeight; W++) dp[0][W] = val[0];
19
20     for(int ind = 1; ind < n; ind++) {
21         for(int W = 0; W <= maxWeight; W++) {
22             int notTake = 0 + dp[ind - 1][W];
23             int take = INT_MIN;
24             if(wt[ind] <= W) {
25                 take = val[ind] + dp[ind-1][W - wt[ind]];
26             }
27             dp[ind][W] = max(take, notTake);
28         }
29     }
30     return dp[n-1][maxWeight];
31     // Write your code here
32 }
33
```

Space optimization -

```
15 int knapsack(vector<int> wt, vector<int> val, int n, int maxWeight)
16 {
17     vector<int> prev(maxWeight+1, 0), cur(maxWeight+1, 0);
18     for(int W = wt[0]; W <= maxWeight; W++) prev[W] = val[0];
19
20     for(int ind = 1; ind < n; ind++) {
21         for(int W = 0; W <= maxWeight; W++) {
22             int notTake = 0 + prev[W];
23             int take = INT_MIN;
24             if(wt[ind] <= W) {
25                 take = val[ind] + prev[W - wt[ind]];
26             }
27             cur[W] = max(take, notTake);
28         }
29         prev = cur;
30     }
31     return prev[maxWeight];
32     // Write your code here

```

further, if we iterate from right to left, we require only 'prev' array.

```
16 {
17     vector<int> prev(maxWeight+1, 0)
18     for(int W = wt[0]; W <= maxWeight; W++) prev[W] = val[0];
19
20     for(int ind = 1; ind < n; ind++) {
21         for(int W = maxWeight; W >= 0; W--) {
22             int notTake = 0 + prev[W];
23             int take = INT_MIN;
24             if(wt[ind] <= W) {
25                 take = val[ind] + prev[W - wt[ind]];
26             }
27             prev[W] = max(take, notTake);
28         }
29
30     }
31     return prev[maxWeight];
32     // Write your code here
33 }
34

```

Minimum Coins-

Problem Statement

[Suggest Edit](#)

You are given an array of 'N' distinct integers and an integer 'X' representing the target sum. You have to tell the minimum number of elements you have to take to reach the target sum 'X'.

Note:

You have an infinite number of elements of each type.

Recursion -

```
1 int f(int ind, int T, vector<int> &nums) {
2     if(ind == 0) {
3         if(T % nums[0] == 0) return T / nums[0];
4         return 1e9;
5     }
6
7     int notTake = 0 + f(ind-1, T, nums);
8     int take = INT_MAX;
9     if(nums[ind] <= T) {
10        a take = 1 + f(ind, T - nums[ind], nums);
11    }
12    return min(take, notTake);
13 }
14 int minimumElements(vector<int> &num, int target)
15 {
16     int n = num.size();
17     int ans = f(n-1, target, num);
18     if(ans >= 1e9) return -1;
19
20 }
```

→ no need to add
f(ind-1, T - nums[ind])
separately
notake will
manage.


```

1  int f(int ind, int T, vector<int> &nums, vector<vector<int>> &dp) {
2      if(ind == 0) {
3          if(T % nums[0] == 0) return T / nums[0];
4          return 1e9;
5      }
6      if(dp[ind][T] != -1) return dp[ind][T];
7      int notTake = 0 + f(ind-1, T, nums, dp);
8      int take = INT_MAX;
9      if(nums[ind] <= T) {
10         take = 1 + f(ind, T - nums[ind], nums, dp);
11     }
12     return dp[ind][T] = min(take, notTake);
13 }
14 int minimumElements(vector<int> &num, int target)
15 {
16     int n = num.size();
17     vector<vector<int>> dp(n, vector<int>(target + 1, -1));
18     int ans = f(n-1, target, num, dp);
19     if(ans >= 1e9) return -1;
20     return ans;
21 }

```

Tabulation-

```

18  for(int T = 0; T <= target; T++) {
19      if(T % nums[0] == 0) dp[0][T] = T / nums[0];
20      else dp[0][T] = 1e9;
21  }
22
23  for(int ind = 1; ind < n; ind++) {
24      for(int T = 0; T <= target; T++) {
25          int notTake = 0 + dp[ind-1][T];
26          int take = INT_MAX;
27          if(nums[ind] <= T) {
28              take = 1 + dp[ind][T - nums[ind]];
29          }
30          dp[ind][T] = min(take, notTake);
31      }
32  }
33
34
35  int ans = dp[n-1][target];
36  if(ans >= 1e9) return -1;
37  return ans;
38 }

```

Unbounded Knapsack problem-(Infinite supply)

You are given 'N' items with certain 'PROFIT' and 'WEIGHT' and a knapsack with weight capacity 'W'. You need to fill the knapsack with the items in such a way that you get the maximum profit. You are allowed to take one item multiple times.

For Example

Recursion -

```
1 int f(int ind, int W, vector<int> &val, vector<int> &wt) {
2     if(ind == 0) {
3         return ((int)(W / wt[0])) * val[0];
4     }
5
6     int notTake = 0 + f(ind-1, W, val, wt);
7     int take = 0;
8     if(wt[ind] <= W) {
9         take = val[ind] + f(ind, W - wt[ind], val, wt);
10    }
11    return max(take, notTake);
12 }
```

Memorization -

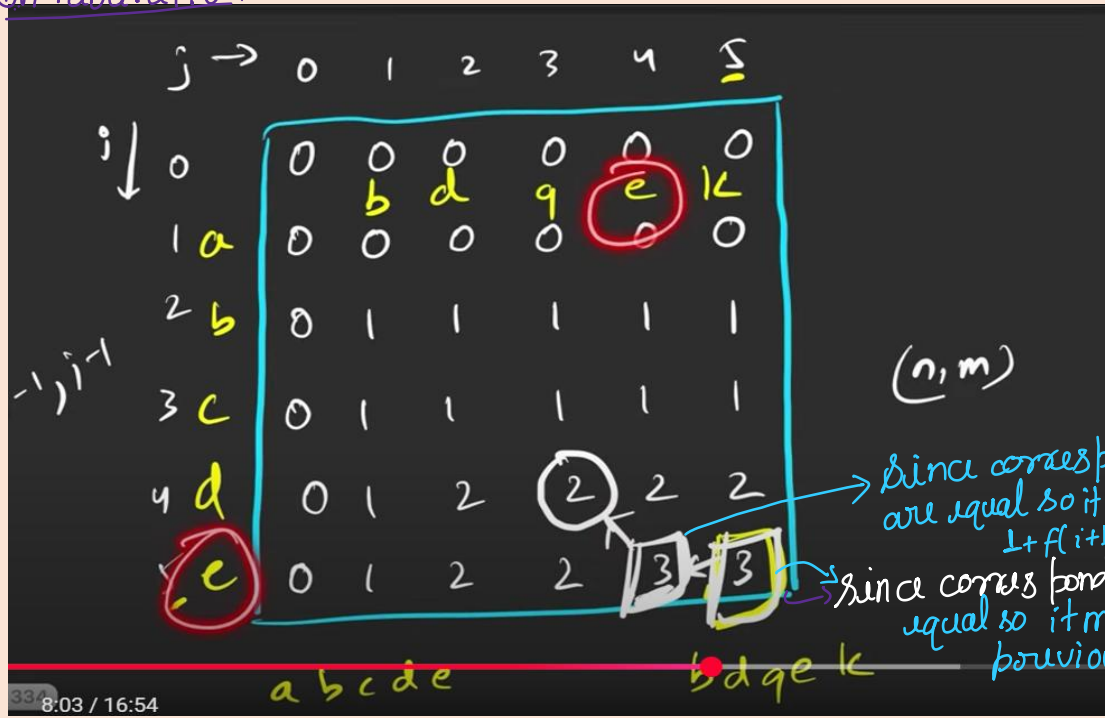
```
1 int f(int ind, int W, vector<int> &val, vector<int> &wt,
2     vector<vector<int>> &dp) {
3     if(ind == 0) {
4         return ((int)(W / wt[0])) * val[0];
5     }
6     if(dp[ind][W] != -1) return dp[ind][W];
7
8     int notTake = 0 + f(ind-1, W, val, wt, dp);
9     int take = 0;
10    if(wt[ind] <= W) {
11        take = val[ind] + f(ind, W - wt[ind], val, wt, dp);
12    }
13    return dp[ind][W] = max(take, notTake);
14 }
15 int unboundedKnapsack(int n, int w, vector<int> &val,
16     vector<int> &wt)
17 {
18     vector<vector<int>> dp(n, vector<int>(w+1, -1));
19     return f(n-1, w, val, wt, dp);
20     // Write Your Code Here.
21 }
```

Tabulation -

```
15 int unboundedKnapsack(int n, int w, vector<int> &val,  
16                       vector<int> &wt)  
17 {  
18     vector<vector<int>> dp(n, vector<int>(w+1, 0));  
19     for(int W = 0; W <= w; W++) {  
20         dp[0][W] = ((int)(W / wt[0])) * val[0];  
21     }  
22  
23     for(int ind = 1; ind < n; ind++) {  
24         for(int W = 0; W <= w; W++) {  
25             int notTake = 0 + dp[ind-1][W];  
26             int take = 0;  
27             if(wt[ind] <= W) {  
28                 take = val[ind] + dp[ind][W - wt[ind]];  
29             }  
30             dp[ind][W] = max(take, notTake);  
31         }  
32     }  
33     return dp[n-1][w];  
34 }
```


Print Longest Common Subsequence-

In tabulation-



```

9   for(int j = 0; j <= m; j++) dp[0][j] = 0; } or declare
10  for(int i = 0; i <= n; i++) dp[i][0] = 0; }
11  for(int i = 1; i <= n; i++) {
12      for(int j = 1; j <= m; j++) {
13          if(s[i-1] == t[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
14          else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
15      }
16  }
17  int len = dp[n][m];
18  string ans = "";
19  for(int i = 0; i < len; i++) {
20      ans += '$';
21  }
22
23  int index = len-1;
24  int i = n, j = m;
25  while(i > 0 && j > 0) {
26      if(s[i-1] == t[j-1]) {
27          ans[index] = s[i-1];
28          index--;
29          i--, j--;
30      }
31      else if(dp[i-1][j] > dp[i][j-1]) {
32          i--;
33      }
34      else j--;
35  }
36  cout << ans;
37
38  }

```

vector<vector<int>>(n+1, vector<int>(m+1, 0))

Longest Common Substring-

↳ consecutive

Handwritten DP table for Longest Common Substring (LCS) between strings "abcd" and "abta".

Strings: *abcd* *abta*

Indices: *j* → *a* *b* *t* *d*

0 *1* *2* *3* *4*

Indices: *i* ↓

<i>0</i>	0	0	0	0	0
<i>a</i> <i>1</i>	0	1	0	0	0
<i>b</i> <i>2</i>	0	0	2	0	0
<i>c</i> <i>3</i>	0	0	0	0	0
<i>d</i> <i>4</i>	0	0	0	0	1

Red circles highlight the values 2 and 0 in the table, indicating the maximum length of the common substring is 2.

```

1  int lcs(string &s, string &t){
2      int n = s.size();
3      int m = t.size();
4      vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
5      for(int j = 0; j<=m; j++) dp[0][j] = 0;
6      for(int i = 0; i<=n; i++) dp[i][0] = 0;
7      int ans = 0;
8      for(int i = 1; i<=n; i++) {
9          for(int j = 1; j<=m; j++) {
10             if(s[i-1] == t[j-1]) {
11                 dp[i][j] = 1 + dp[i-1][j-1];
12                 ans = max(ans, dp[i][j]);
13             }
14             else dp[i][j] = 0;
15         }
16     }
17     return ans;
18 }

```

Longest Palindromic Subsequence-

Take the opposite of the given string (S). Suppose that string is S_1 , Now find longest common subsequence of S & S_1 . That will be the longest palindromic subsequence.

✓ Minimum Insertions Required to make a string

palindrome- Suppose we have string 'abc aa', in order to make it palindromic, we first extract the longest palindromic subsequence from this string then put the ^{rest} non-palindromic subsequence in a reverse order.

abc a cba

So, minimum insertions required = length of string - size of longest palindromic subsequence.

Minimum Insertions/Deletion Required to Convert a String

A to String B- (We can insert anywhere/any character in/of string A. $\downarrow$ length of string A.)

(x) No. of deletions from string A = $n - \text{length of longest common subsequence}$.

(y) No. of insertions = $m - \text{length of longest common subsequence}$.
 $\uparrow$ length of string B

So, Total operations = $x + y$.

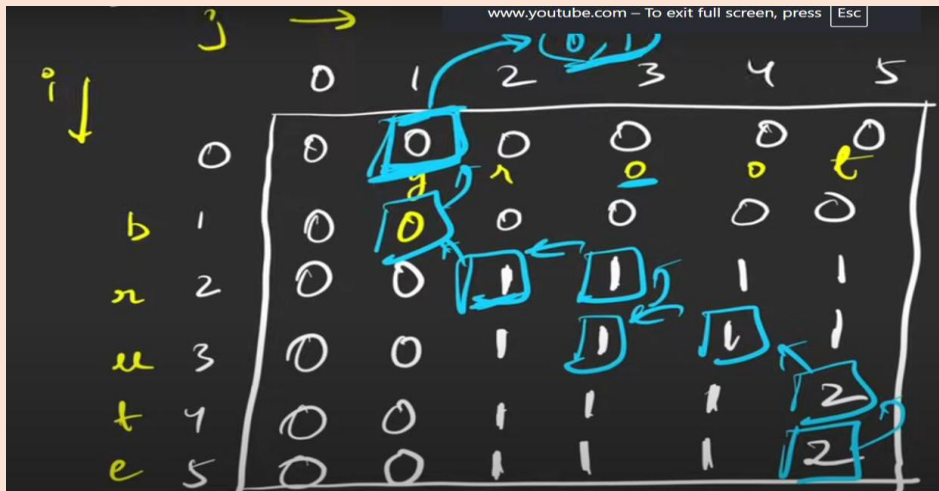
Shortest Common Supersequence-

$S_1 = \text{brute}$, $S_2 = \text{groot}$.

supersequence = bgroote.

→ length of shortest common supersequence = $n + m - \text{length of longest common subsequence}$.

To print the supersequence -



```
while(i > 0 && j > 0) {  
    if(s1[i-1] == s2[j-1]) {  
        ans += s1[i-1];  
        i--;  
        j--;  
    }  
    else if(dp[i-1][j] > dp[i][j-1]) {  
        ans += s1[i-1];  
        i--;  
    }  
    else {  
        ans += s2[j-1];  
        j--;  
    }  
}  
while(i > 0) ans += s1[i-1], i--;  
while(j > 0) ans += s2[j-1], j--;  
rev(ans);
```

Distinct Subsequences-

Given a string S1 and String S2 find the number of occurrences of string S2 in string S1.

$S_1 = \text{bagabagbag}$, $S_2 = \text{bag}$

Recursion -

$f(i, j) \{$

$\text{if}(j < 0) \text{return } 1;$

$\text{if}(i < 0) \text{return } 0;$

$\text{if}(S_1[i] = S_2[j]) \text{return } f(i-1, j-1) + f(i-1, j);$

$\text{else return } f(i-1, j);$

$\}$

```
16 int numDistinct(string s, string t) {
17     int n = s.size();
18     int m = t.size();
19     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
20
21     for(int i = 0; i <= n; i++) dp[i][0] = 1;
22     for(int j = 1; j <= m; j++) dp[0][j] = 0;
23
24     for(int i = 1; i <= n; i++) {
25         for(int j = 1; j <= m; j++) {
26             if(s[i-1] == t[j-1]) {
27                 dp[i][j] = dp[i-1][j-1]
28                     + dp[i-1][j];
29             }
30             else {
31                 dp[i][j] = dp[i-1][j];
32             }
33         }
34     }
35     return dp[n][m];
36 }
```

→ gn ($i < 0$ OR $j < 0$) in recursion, to avoid negative value in tabulation we added +1 to everything, but while accessing elements taken 0-indexed.

EDIT DISTANCE-

You are given two strings S1 and S2. You can perform three operations-Insert, Remove, Replace. Find the minimum number of steps required to convert the given String S1 to String S2.

Ex. $S_1 = \text{horse}$, $S_2 = \text{ros}$

$f(i, j)$
 if $(i < 0)$ return $j+1$; $\rightarrow$ means if string 1 is exhausted then still in order to convert S_1 to S_2 we need $j+1$ insert operations.
 if $(j < 0)$ return $i+1$; $\rightarrow$ if S_2 is exhausted then we need to delete the rest $i+1$ of S_1 .

$f(S1[i] == S2[j])$ return $0 + f(i-1, j-1)$

$\min \begin{cases} 1 + f(i, j-1) & // \text{insert} \\ 1 + f(i-1, j) & // \text{delete} \\ 1 + f(i-1, j-1) & // \text{replace} \end{cases}$

Memorization-

```
int f(int i, int j, string &s1, string &s2,
    vector<vector<int>> &dp) {
    2 if( i < 0) return j+1;
    3 if( j < 0) return i+1;
    4 if(dp[i][j] != -1) return dp[i][j];
    5 if(s1[i] == s2[j]) return dp[i][j] = f(i-1, j-1, s1, s2, dp);
    6 return dp[i][j] = 1 + min(f(i-1, j, s1, s2, dp),
    7 min(f(i, j-1, s1, s2, dp),
    8 f(i-1, j-1, s1, s2, dp)));
    9 }
10
11 int editDistance(string str1, string str2)
12 {
13     int n = str1.size();
14     int m = str2.size();
15     vector<vector<int>> dp(n, vector<int>(m, -1));
16     return f(n-1, m-1, str1, str2, dp);
17     //write you code here
18 }
```

Tabulation-

```

14 int m = str2.size();
15 vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
16 for(int i = 0; i <= n; i++) dp[i][0] = i;
17 for(int j = 0; j <= m; j++) dp[0][j] = j;
18
19 for(int i = 1; i <= n; i++) {
20     for(int j = 1; j <= m; j++) {
21         if(s1[i-1] == s2[j-1])
22             dp[i][j] = dp[i-1][j-1];
23         else dp[i][j] = 1 + min(dp[i-1][j],
24                                 min(dp[i][j-1], dp[i-1][j-1]));
25     }
26 }
27
28 return dp[n][m];
29 //write you code here
30 }

```

Wildcard Matching

? → matches with single character of S_2 (? → will be present only in S_1).

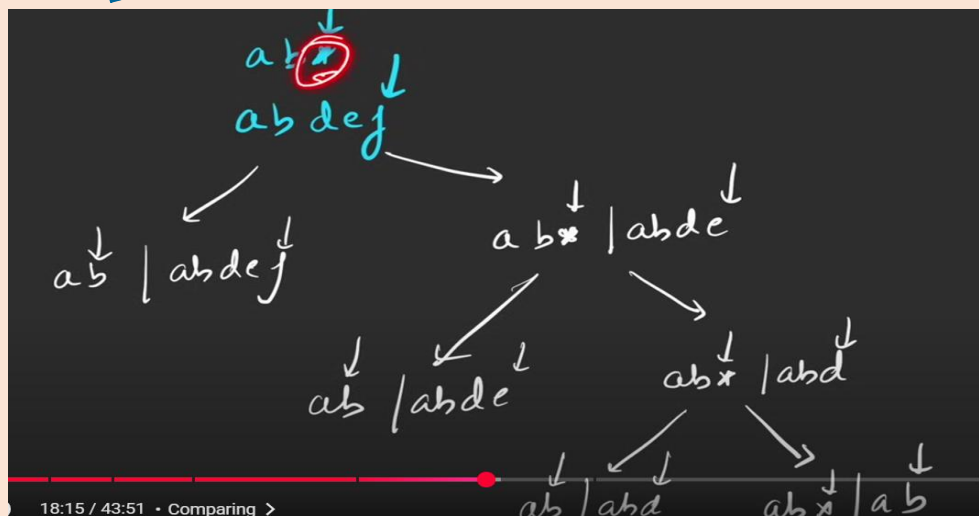
* matches with sequence of length 0 or more.

Ex. $S_1 = ?ag.$

$S_2 = ray$

all three characters of S_1 matches with S_2 .

Ex. $S_1 = abcd*$
 $S_2 = abcdef$ } → all matches, so True.



$f(i, j) \{$
 if $(i < 0 \ \& \ j < 0)$ return true;

if (if $i < 0$ & $j > 0$) return false;
 if ($j < 0$ & $i > 0$) { if {for (int $x = 0$; $x \leq i$, $x++$) { $s_1[x] = '*'$ } } return true;
 ↳ means if s_2 is exhausted means $s_2 = ""$. then, to give true results just characters in s_1 has to be '*'.

Recursion -

if ($s_1[i] == s_2[j]$ || $s_1[i] == '?'$)

return $f(i-1, j-1)$;

if ($s_1[i] == '*'$)

return

$f(i-1, j)$ or $f(i, j-1)$

return false;

Dp On Stocks-

1). Best Time To Buy and sell stocks- You are given an array which tells the stock prices of different days, You have to first buy the stock before selling it, tell the maximum profit you can generate.

Note that - If you are selling stocks on the i^{th} day, then you will have to buy it between 1^{st} to $(i-1)^{\text{th}}$ day.

```
mini = a[0], profit = 0
for (i = 1; i < n; i++)
{
    cost = a[i] - mini;
    profit = max(profit, cost);

    mini = min(mini, a[i]);
}
```

2). Best time to buy and sell stocks-II (We can buy and sell multiple times) → Note that once you buy then before performing the next buy you will have to sell the first one.

Recursion.

→ If 1, we can buy if 0, we cannot buy.
(It will be used to make sure that after buy there cannot be another buy before selling.)

```
if (idx == n) return 0;

if (buy) {
    profit = max(
        -prices[idx] + f(idx+1, 0) ← bought (so, profit = sell - bought)
        0 + f(idx+1, 1) ← not take.
    )
}

// performed buy so further cannot buy.
```

$$\text{profit} = \max \left(\begin{array}{l} \text{prices}[\text{idx}] + f(\text{idx}+1, 1) \\ 0 + f(\text{idx}+1, 0) \end{array} \right)$$
 ↖ sold.

}
 return profit;

Tabulation

```

18 {
19     vector<vector<long>> dp(n+1, vector<long>(2, 0));
20
21     dp[n][0] = dp[n][1] = 0;
22     for(int ind = n-1; ind >= 0; ind--) {
23         for(int buy = 0; buy <= 1; buy++) {
24             long profit = 0;
25             if(buy) {
26                 profit = max(-values[ind] + dp[ind+1][0],
27                             0 + dp[ind+1][1]);
28             }
29             else {
30                 profit = max(values[ind] + dp[ind+1][1],
31                             0 + dp[ind+1][0]);
32             }
33             dp[ind][buy] = profit;
34         }
35     }
36     return dp[0][1];
37     //Write your code here
38 }
  
```

Space optimi

```

17 long getMaximumProfit(long *values, int n)
18 {
19     vector<long> ahead(2, 0), cur(2, 0);
20
21     ahead[0] = ahead[1] = 0;
22     for(int ind = n-1; ind >= 0; ind--) {
23         for(int buy = 0; buy <= 1; buy++) {
24             long profit = 0;
25             if(buy) {
26                 profit = max(-values[ind] + ahead[0],
27                             0 + ahead[1]);
28             }
29             else {
30                 profit = max(values[ind] + ahead[1],
31                             0 + ahead[0]);
32             }
33             cur[buy] = profit;
34         }
35         ahead = cur;
36     }
37     return ahead[1];
38 }
  
```


3)- Best time to buy and sell stocks -III

(At max 2 transactions means 2 times Buy and sell).

Recursive:-

$f(idx, buy, cap)$ {
 $\nearrow$ max transactions that can be done.

if ($cap == 0$) return 0;

if ($idx == n$) return 0;

if (buy) {

profit = max $\left(\begin{array}{l} -prices[idx] + f(idx+1, 0, cap) \\ 0 + f(idx+1, 1, cap) \end{array} \right)$

it will not be
reduced as 1 transact
means Buy + sell.

}

else {

profit = max $\left(\begin{array}{l} prices[idx] + f(idx+1, 1, cap-1) \\ 0 + f(idx+1, 0, cap) \end{array} \right)$

}

return profit;

}

Memorization-

```
1 int f(int ind, int buy, int cap, vector<int> &prices, int n,
2     vector<vector<vector<int>>> &dp) {
3     if(ind == n || cap == 0) return 0;
4     if(dp[ind][buy][cap] != -1) return dp[ind][buy][cap];
5     if(buy == 1) {
6         return dp[ind][buy][cap] =
7             max(-prices[ind] + f(ind + 1, 0, cap, prices, n),
8                 0 + f(ind+1, 1, cap, prices, n));
9     }
10    return dp[ind][buy][cap] = max(prices[ind] + f(ind + 1, 1, cap - 1),
11        0 + f(ind+1, 0, cap, prices, n));
12
13 }
14 int maxProfit(vector<int>& prices, int n)
15 {
16     vector<vector<vector<int>>> dp(n,
17         vector<vector<int>> (2, vector<int> (3, -1)));
18     return f(0, 1, 2, prices, n, dp);
19     // Write your code here.
20 }
```

Tabulation-

```
15 int maxProfit(vector<int>& prices, int n)
16 {
17     vector<vector<vector<int>>> dp(n+1,
18         vector<vector<int>> (2, vector<int> (3, 0)));
19
20     for(int ind = n - 1; ind >= 0; ind--) {
21         for(int buy = 0; buy <= 1; buy++) {
22             for(int cap = 1; cap <= 2; cap++) {
23                 if(buy == 1) {
24                     dp[ind][buy][cap] =
25                         max(-prices[ind] + dp[ind + 1][0][cap],
26                             0 + dp[ind+1][1][cap]);
27                 }
28                 else
29                     dp[ind][buy][cap] =
30                         max(prices[ind] + dp[ind + 1][1][cap - 1],
31                             0 + dp[ind+1][0][cap]);
32             }
33         }
34     }
35     return f(0, 1, 2, prices, n, dp); dp[0][1][2];
```

Space Optimization-

```
15 int maxProfit(vector<int> &prices, int n)
16 {
17
18     vector<vector<int>> after(2, vector<int> (3, 0));
19     vector<vector<int>> cur(2, vector<int> (3, 0));
20
21     for(int ind = n - 1; ind >= 0; ind--) {
22         for(int buy = 0; buy <= 1; buy++) {
23             for(int cap = 1; cap <= 2; cap++) {
24                 if(buy == 1) {
25                     cur[buy][cap] =
26                         max(-prices[ind] + after[0][cap],
27                             0 + after[1][cap]);
28                 }
29                 else
30                     cur[buy][cap] =
31                         max(prices[ind] + after[1][cap - 1],
32                             0 + after[0][cap]);
33             }
34         }
35     }
```

after = cur;
return after[1][2];

4)- Buy and Sell stocks-IV → k-transactions are allowed.
→ similar to above, just value of cap is k instead of 2.

5) Best time to buy and sell after cooldown- (Means cannot buy right after sell).

$f(\text{ind}, \text{buy}) \{$

```

if (ind == n) return 0;

if (buy)
    return mem { price[ind] + f(ind+1, 0);
                0   + f(ind+1, 1);
            }
else
    return mem { price[ind] + f(ind+1ind+2, 1);
                0   + f(ind+1, 0);
            }
    }

```

6)- Buy and sell stocks after transaction fee-

```

f(ind, buy)
{
    if (ind == 0) return 0;

    if (buy == 1)
        mem { price[ind] + f(ind+1, 0);
              0   + f(ind+1, 1);
            }
    else
        mem { price[ind] - fee + f(ind+1, 1);
              0   + f(ind+1, 0);
            }
}

```

OR subtract fee from here.

BS → 11

Ques- Find length of Longest Increasing Subsequence-

arr[] $\rightarrow$ [10, 9, 2, 5, 3, 7, 101, 18]

{2, 3, 7, 101} $\rightarrow$ len 4
{2, 3, 7, 18} $\rightarrow$ len 4 } and increasing.

Recursion-

```
1 int f(int ind, int prev_ind, int arr[], int n) {  
2     if(ind == n) return 0;  
3  
4     int len = 0 + f(ind+1, prev_ind, arr, n);  
5     if(prev == -1 || arr[ind] > arr[prev_ind]) {  
6         len = max(len, 1 + f(ind+1, ind, arr, n));  
7     }  
8     return len;  
9 }  
10 int longestIncreasingSubsequence(int arr[], int n)  
11 {  
12     return f(0, -1, arr, n);  
13 }  
14
```

Memorization-

```
1 int f(int ind, int prev_ind, int arr[], int n,  
2     vector<vector<int>> &dp) {  
3     if(ind == n) return 0;  
4     if(dp[ind][prev_ind+1] != -1) return dp[ind][prev_ind+1];  
5     int len = 0 + f(ind+1, prev_ind, arr, n);  
6     if(prev_ind == -1 || arr[ind] > arr[prev_ind]) {  
7         len = max(len, 1 + f(ind+1, ind, arr, n));  
8     }  
9     return dp[ind][prev_ind+1] = len;  
10 }  
11 int longestIncreasingSubsequence(int arr[], int n)  
12 {  
13     vector<vector<int>> dp(n, vector<int>(n+1, -1));  
14     return f(0, -1, arr, n, dp);  
15 }  
16
```

$\rightarrow$ as here prev-ind is being used as ind of dp so instead of putting its value as -1 we add 1 to it.

